PRACTICAL 6

Aim :

Connect the React frontend to the Node.js, Express, and MongoDB backend. Implement a complete task workflow in the React interface so users can create, view, update, and delete tasks through API calls, with loading and error handling for every operation.

Code :

frontend/vite.config.js :-

```
import { defineConfig } from 'vite';

import react from '@vitejs/plugin-react';



export default defineConfig(({ mode }) => {

  const apiTarget = process.env.BACKEND_URL || 'http://localhost:5500';

  return {

    plugins: [react()],

    server: {

      proxy: {

        '/api': {

          target: apiTarget,

          changeOrigin: true,

          secure: false

        }

      }

    }

  };

});
```

frontend/src/api.js :-

```
const API_BASE = import.meta.env.VITE_API_BASE || '/api';


async function request(path, options = {}) {
  const response = await fetch(`${API_BASE}${path}`, {
    headers: { 'Content-Type': 'application/json', ...(options.headers || {}) },
    ...options
  });
  const data = await response.json().catch(() => ({}));
  if (!response.ok) throw new Error(data.error || 'Request failed');
  return data;
}


export const getTasks = () => request('/tasks');
export const createTask = (task) => request('/tasks', {
  method: 'POST', body: JSON.stringify(task)
});
export const updateTask = (id, task) => request(`/tasks/${id}`, {
  method: 'PUT', body: JSON.stringify(task)
});
export const deleteTask = (id) => request(`/tasks/${id}`, { method: 'DELETE' });
```


Code :

frontend/src/pages/Projects.jsx :-

```
import { useEffect, useState } from 'react';
import Spinner from '../components/Spinner';
import ErrorMessage from '../components/ErrorMessage';
import { createTask, deleteTask, getTasks, updateTask } from '../api';


export default function Projects() {
```

```
 const [tasks, setTasks] = useState([]);
 const [loading, setLoading] = useState(true);
 const [error, setError] = useState(null);
 const [actionError, setActionError] = useState(null);
 const [saving, setSaving] = useState(false);

 function fetchTasks() {
  setLoading(true);
  setError(null);
  getTasks().then(setTasks)
    .catch((err) => setError(err.message))
    .finally(() => setLoading(false));
 }

 useEffect(() => { fetchTasks(); }, []);

 function handleCreate(event) {
  event.preventDefault();
  setSaving(true);
  createTask({ title, description, priority })
    .then((task) => setTasks((current) => [task, ...current]))
    .catch((err) => setActionError(err.message))
    .finally(() => setSaving(false));
 }

 function saveEdit(id) {
  setSaving(true);
  updateTask(id, { title: editTitle, description: editDescription })
    .then((task) => setTasks((current) => current.map((item) =>
      item._id === id ? task : item)))
    .catch((err) => setActionError(err.message))
    .finally(() => setSaving(false));
 }

 function handleDelete(id) {
  if (!window.confirm('Delete this task?')) return;
  setSaving(true);
  deleteTask(id)
    .then(() => setTasks((current) => current.filter((task) => task._id !== id)))
```

```
    .catch((err) => setActionError(err.message))
    .finally(() => setSaving(false));
 }

 return (
```

Projects / Tasks

```
    {/* Form, loading/error states, task list, edit, complete, and delete controls */}
 );
}
```

Backend API used :-

GET /api/tasks - read all tasks

POST /api/tasks - create a task

PUT /api/tasks/:id - update a task

DELETE /api/tasks/:id - delete a task

Testing :

Run both applications in separate terminals. The backend must be connected to MongoDB before the frontend sends task requests.

Terminal 1:

cd backend

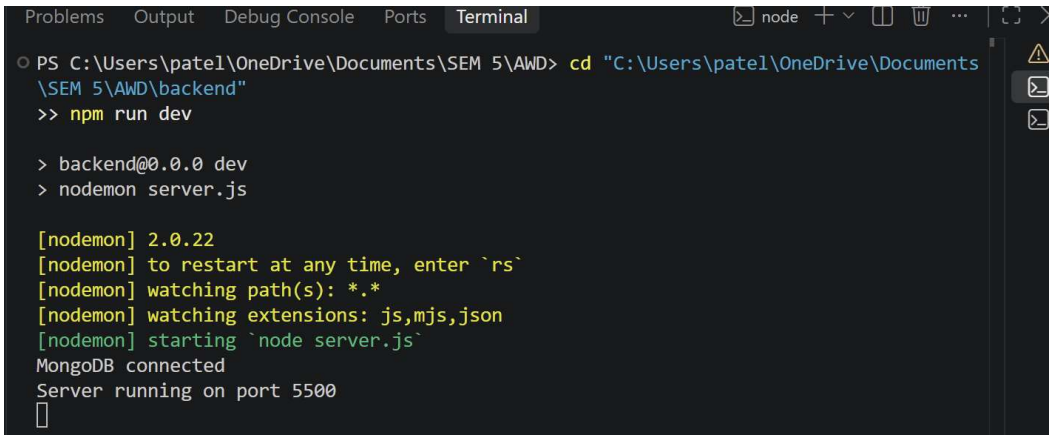
npm run dev

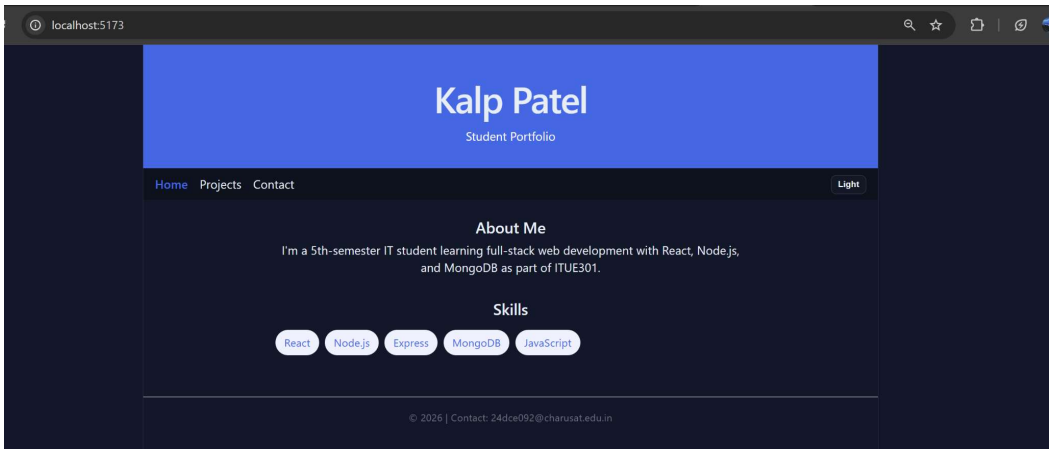
Terminal 2:

cd frontend

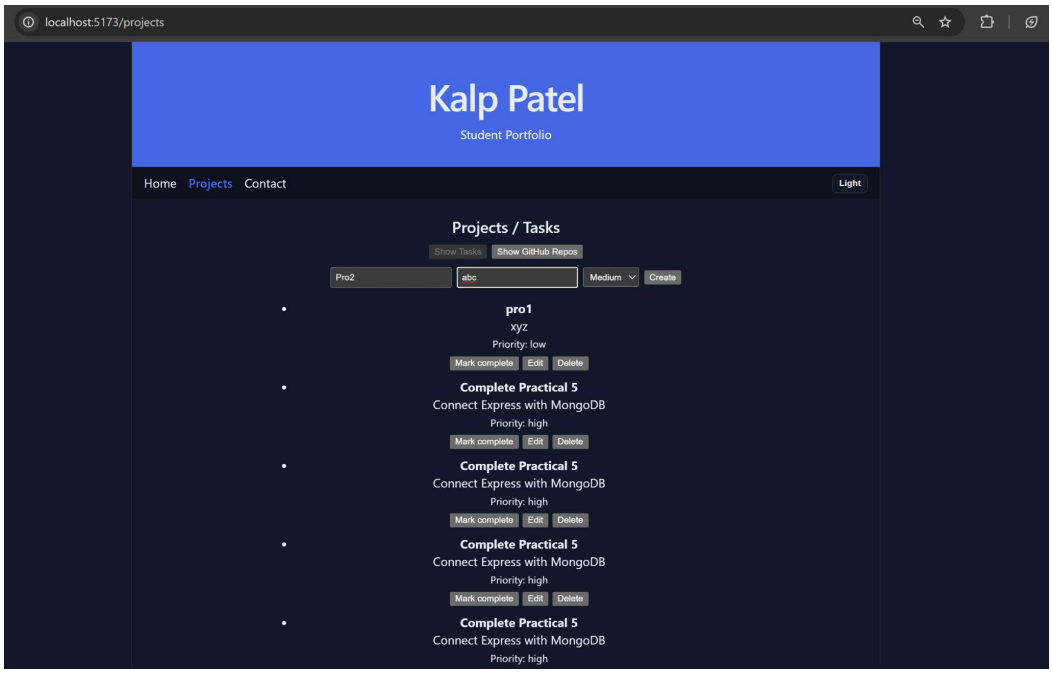
npm run dev

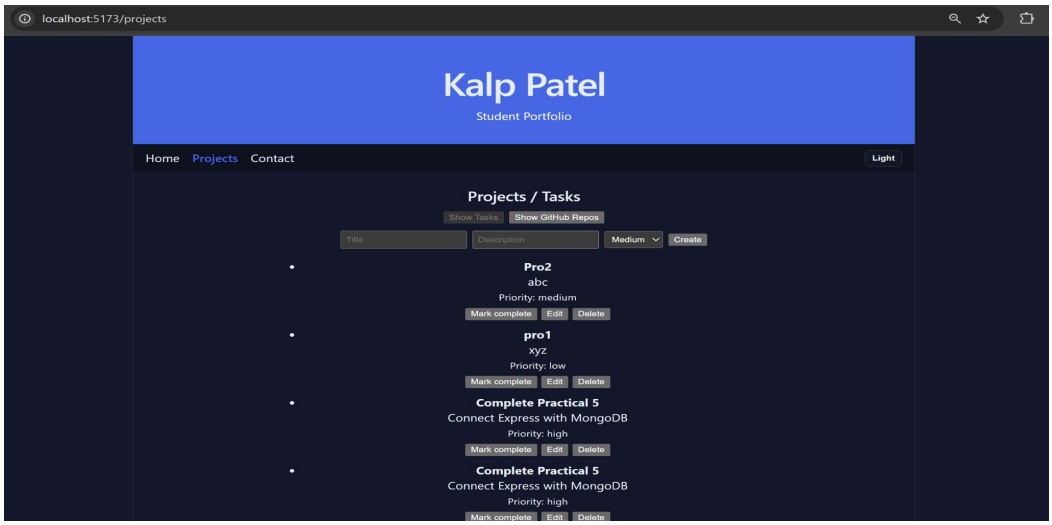
Output :

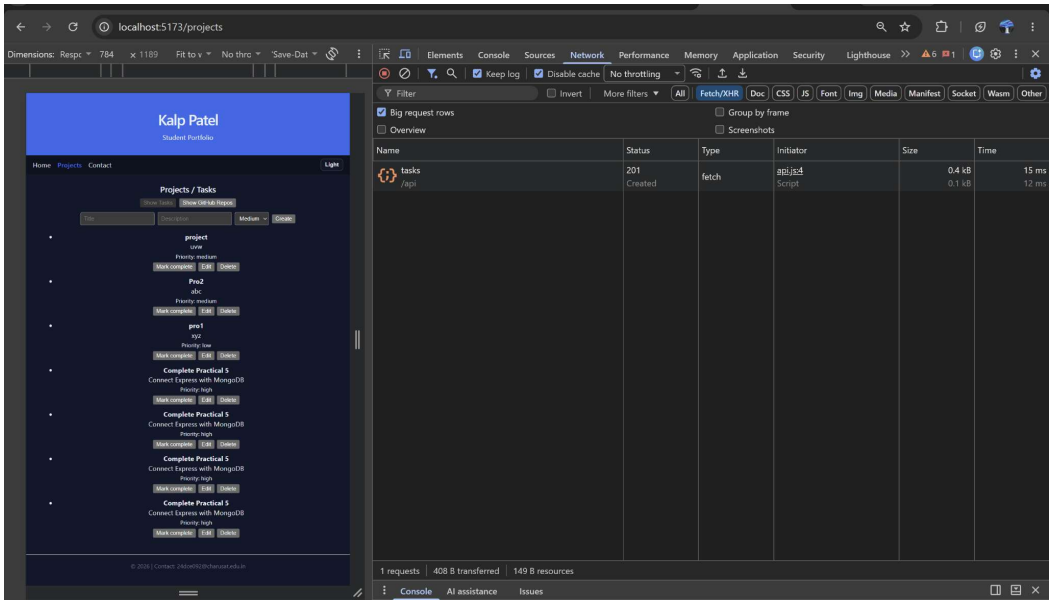


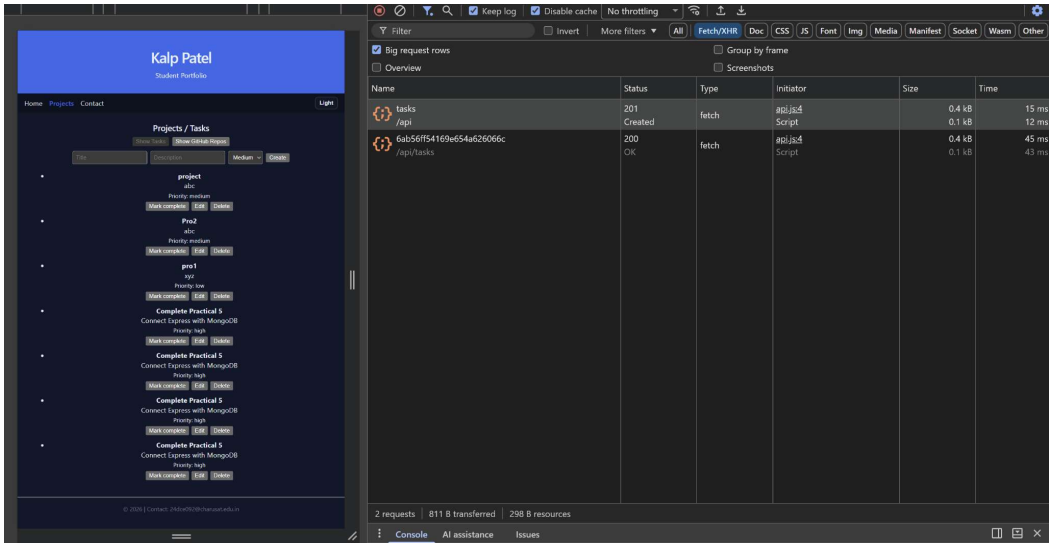

localhost:5173
Kalp Patel
Student Portfolio
Home Projects Contact
Light
About Me
I'm a 5th-semester IT student learning full-stack web development with React, Node.js, and MongoDB as part of ITUE301.
Skills
React Node.js Express MongoDB JavaScript
© 2026 | Contact: 24dce092@charusat.edu.in

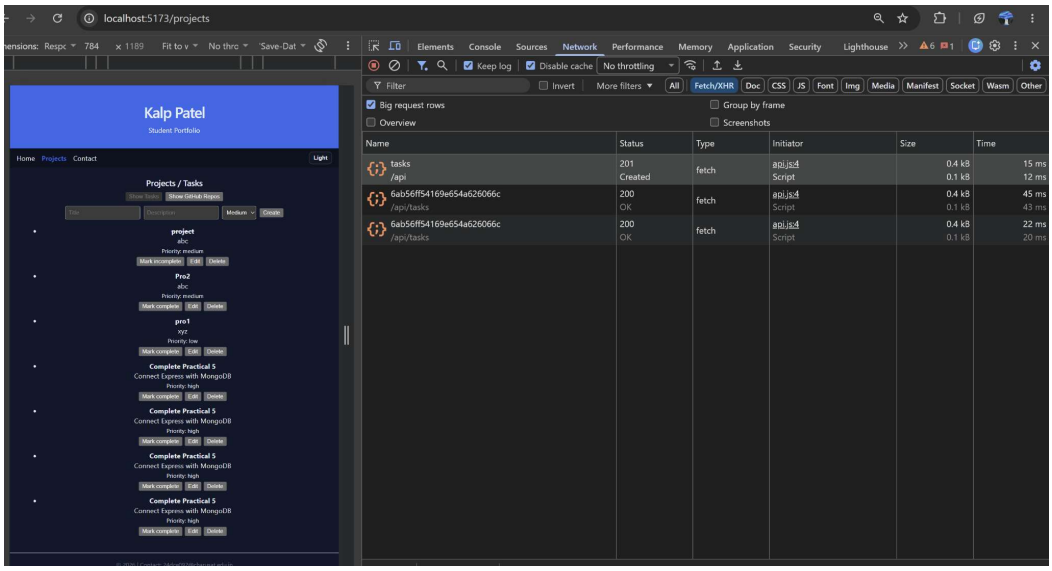
Output :

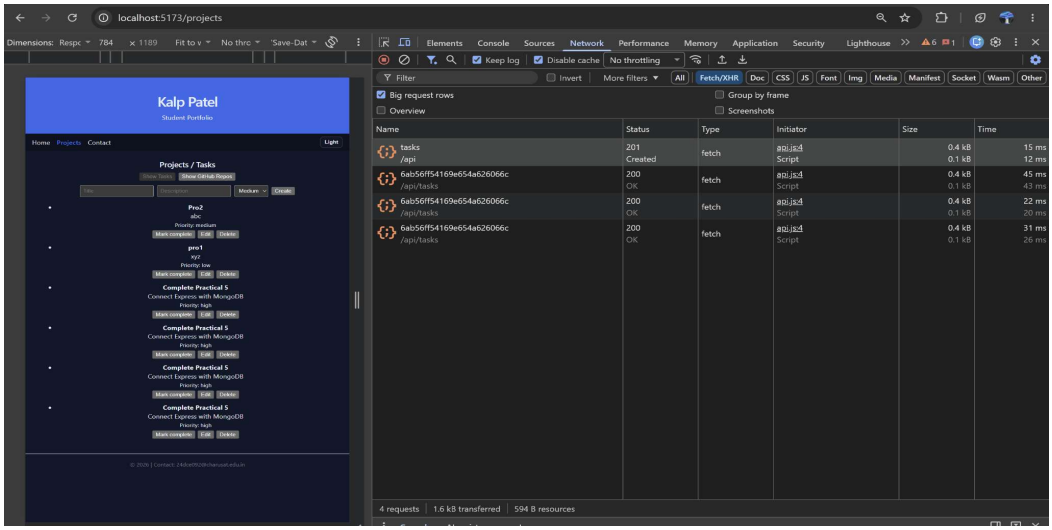

localhost:5173/projects
Kalp Patel
Student Portfolio
Home Projects Contact
Light
Projects / Tasks
Show Tasks
Show GitHub Repos
Pro2
abc
Medium
Create
pro1
xyz
Priority: low
Mark complete Edit Delete
Complete Practical 5
Connect Express with MongoDB
Priority: high
Mark complete Edit Delete
Complete Practical 5
Connect Express with MongoDB
Priority: high
Mark complete Edit Delete
Complete Practical 5
Connect Express with MongoDB
Priority: high
Mark complete Edit Delete
Complete Practical 5
Connect Express with MongoDB
Priority: high

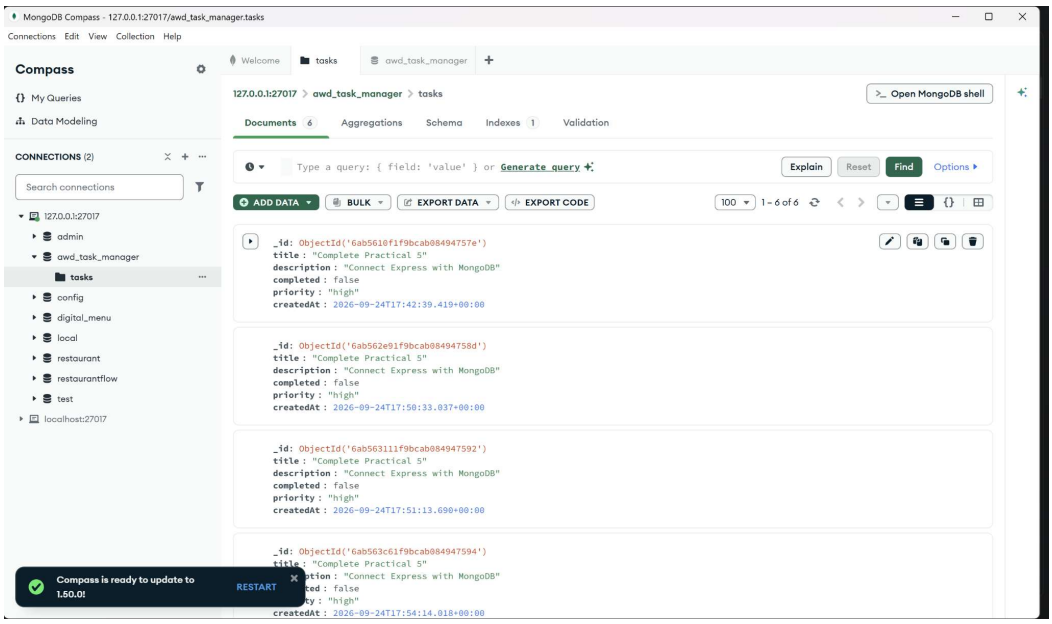

localhost:5173/projects
Kalp Patel
Student Portfolio
Home Projects Contact
Light
Projects / Tasks
Show Tasks
Show GitHub Repos
Title
Description
Medium
Create
Pro2
abc
Priority: medium
Mark complete
Edit
Delete
pro1
xyz
Priority: low
Complete Practical 5
Connect Express with MongoDB
Priority: high


localhost:5173/projects
Elements
Console
Sources
Network
Performance
Memory
Application
Security
Lighthouse
Keep log
Disable cache
No throttling
Filter
Invert
More filters
Fetch/XHR
Big request rows
Group by frame
Overview
Screenshots
Name
Status
Type
Initiator
Size
Time
tasks
/api
201
Created
fetch
api.js:4
Script
0.4 kB
0.1 kB
15 ms
12 ms
1 requests
408 B transferred
149 B resources
Console
AI assistance
Issues


Kalp Patel
Student Portfolio
Projects / Tasks
Keep log
Disable cache
No throttling
Filter
Invert
More filters
Fetch/XHR
Big request rows
Group by frame
Overview
Screenshots
Name
Status
Type
Initiator
Size
Time
tasks
/api
201
Created
fetch
api.js:4
Script
0.4 kB
0.1 kB
15 ms
12 ms
6ab56ff54169e654a626066c
/api/tasks
200
OK
fetch
api.js:4
Script
0.4 kB
0.1 kB
45 ms
43 ms
2 requests
811 B transferred
298 B resources
Console
AI assistance
Issues


localhost:5173/projects
Kalp Patel
Student Portfolio
Projects / Tasks
Elements
Console
Sources
Network
Performance
Memory
Application
Security
Lighthouse
Name
Status
Type
Initiator
Size
Time


localhost:5173/projects
Kalp Patel
Student Portfolio
Projects / Tasks
Elements
Console
Sources
Network
Performance
Memory
Application
Security
Lighthouse
Name
Status
Type
Initiator
Size
Time
4 requests | 1.6 kB transferred | 594 B resources


MongoDB Compass - 127.0.0.1:27017/awd_task_manager.tasks
Compass
My Queries
Data Modeling
CONNECTIONS (2)
Search connections
127.0.0.1:27017
admin
awd_task_manager
tasks
config
digital_menu
local
restaurant
restaurantflow
test
localhost:27017
Documents 6
Aggregations
Schema
Indexes 1
Validation
Open MongoDB shell
ADD DATA
EXPORT DATA
EXPORT CODE
Compass is ready to update to 1.50.0!
RESTART

Result :

The React frontend was successfully integrated with the Express and MongoDB backend. Tasks can be created, read, updated, and deleted from the user interface. The UI displays loading and error states for API operations, updates its state from server responses, and retains task data after refreshing the browser.